

Lección 21: OWASP Top 10 Aplicado

Vulnerabilidades Críticas en Producción

Agenda

- OWASP Top 10 2021: Resumen
 - Las 7 Vulnerabilidades Más Críticas:
 - A01: Control de Acceso Roto
 - A02: Fallas Criptográficas
 - A03: Inyección
 - A04: Diseño Inseguro
 - A05: Configuración de Seguridad Incorrecta
 - A07: Fallas de Autenticación
 - A09: Fallas en Logging y Monitoreo
-

OWASP Top 10: ¿Qué es?

Open Web Application Security Project

Top 10 vulnerabilidades más críticas en aplicaciones web

- Actualizado cada 3-4 años
- Basado en datos de seguridad real
- Estándar de la industria
- Guía para developers y security teams

Versión actual: OWASP Top 10 2021

¿Por qué importa?

- 94% de aplicaciones tienen al menos una vulnerabilidad Top 10
 - Responsable de la mayoría de brechas de seguridad
 - Fácilmente explotables
 - Alto impacto en negocio
-

¿Por qué solo 7 de 10?

Cubrimos en esta lección (P0 - Crítico):

- A01, A02, A03, A04, A05, A07, A09

No cubrimos (menos comunes en frontends):

A06: Componentes Vulnerables

→ Ya cubierto: `npm audit` (Lección 19)

A08: Fallas de Integridad

→ Avanzado: `supply chain attacks`, `unsigned deployments`

A10: SSRF (Server-Side Request Forgery)

→ Backend-only: server hace requests a URLs controladas por atacante

Enfoque: Vulnerabilidades que afectan directamente tu shopping cart app

A01: Control de Acceso Roto (Broken Access Control)

El Problema:

Usuario normal puede acceder a recursos de admin

Usuario puede ver/modificar datos de otros usuarios

Bypass de permisos mediante manipulación de URL

✗ Vulnerable:

// Validación solo del lado del cliente

```
<button disabled={!user.isAdmin}>
```

```
Delete User  
</button>
```

```
// Backend sin validación  
app.delete('/api/users/:id', async (req, res) => {  
  await db.users.delete(req.params.id) // Cualquiera puede llamarlo!  
  res.json({ success: true })  
})
```

A01: Defensa

Regla de Oro: Verificar SIEMPRE en el backend

 **MAL:** Validación solo en frontend
`<button disabled={!isAdmin}>Delete</button>`
→ Atacante llama API directamente

 **BIEN:** Backend valida en cada request

1. ¿Usuario autenticado? → Token válido
2. ¿Tiene permisos? → Verificar rol (admin/user)
3. ¿Es dueño del recurso? → userId coincide

Ejemplo Práctico:

Usuario intenta ver pedido #123:

1. Backend verifica: ¿Token válido? 
2. Backend verifica: ¿Pedido pertenece a este usuario? 
3. Si NO es su pedido → 403 Forbidden
4. Si es admin → Puede ver todos los pedidos

A02: Fallas Criptográficas (Cryptographic Failures)

El Problema:

Passwords almacenados en texto plano
Datos sensibles sin encriptar en tránsito

Algoritmos débiles (MD5, SHA1)
Secrets hardcoded en código

❌ Vulnerable:

```
// Password sin hashear
await db.users.create({
  email: req.body.email,
  password: req.body.password, // Texto plano!
})

// HTTP sin HTTPS
fetch('http://api.example.com/payment', {
  body: JSON.stringify({
    cardNumber: '4111111111111111',
    cvv: '123',
  }),
})
```

A02: Defensa

3 Reglas Clave:

1. NUNCA guardar passwords en texto plano

❌ MAL: password: "12345678"
✅ BIEN: passwordHash: "bcrypt hash generado"

Usar bcrypt con 12+ rounds (lento = seguro)

2. SIEMPRE usar HTTPS

HTTP → Datos visibles en la red ❌
HTTPS → Datos encriptados ✅

Forzar redirect: http://... → https://...
Header HSTS: Browser recuerda usar HTTPS

3. Encriptar datos sensibles en DB

Sensibles: Tarjetas de crédito, SSN, datos médicos
Librería: crypto (Node.js) o equivalente
Clave de encriptación: En variable de entorno

A03: Inyección (Injection) (1/2)

El Problema:

SQL, NoSQL, LDAP, OS command injection
Input de usuario no sanitizado ejecutado como código

✗SQL Injection:

```
// Concatenación de strings
const email = req.query.email
const query = `SELECT * FROM users WHERE email = '${email}'`
const users = await db.query(query)

// Atacante envía: ' OR '1'='1
// Query: SELECT * FROM users WHERE email = '' OR '1'='1'
// Retorna TODOS los usuarios!
```

A03: Inyección (Injection) (2/2)

✗NoSQL Injection:

```
// MongoDB vulnerable
db.users.find({
  email: req.body.email,
  password: req.body.password,
})

// Atacante envía: { "$ne": null }
// Bypasea verificación de password!
```

A03: Defensa

Regla de Oro: NUNCA concatenar input de usuario en queries

- ❌ MAL: Concatenación de strings
query = "SELECT * FROM users WHERE email = '" + email + "'"
- Atacante envía: ' OR '1'='1
- Query ejecuta código malicioso

- ✅ BIEN: Prepared statements (queries parametrizadas)
query = "SELECT * FROM users WHERE email = ?"
- db.query(query, [email])
- Driver trata email como DATA (no código)
- Inyección imposible

3 Capas de Defensa:

1. Usar ORM/prepared statements (Prisma, TypeORM)
2. Validar tipos con Zod (rechaza caracteres sospechosos)
3. Sanitizar input (express-mongo-sanitize para NoSQL)

A04: Diseño Inseguro (Insecure Design) (1/2)

El Problema:

Fallas en arquitectura y diseño
Falta de modelado de amenazas (threat modeling)
Sin requisitos de seguridad

A04: Diseño Inseguro (Insecure Design) (2/2)

❌ Ejemplos de Diseño Inseguro:

```
// Intentos ilimitados de reseteo de password
app.post('/forgot-password', async (req, res) => {
  const token = generateResetToken()
```

```
    await sendEmail(req.body.email, token)
    res.json({ success: true })
  })
  // → Atacante puede hacer brute force de tokens

  // Sin rate limiting en login
  app.post('/login', async (req, res) => {
    const user = await verifyCredentials(req.body)
    res.json({ token: createToken(user) })
  })
  // → Atacante puede probar millones de passwords

  // Retornar errores sensibles
  if (!user) {
    return res.status(401).json({
      error: 'User not found', // Fuga de información!
    })
  }
}
```

A04: Defensa

Principio: Defensa en profundidad (múltiples capas de protección)

- ✗ MAL: Sin límites
Usuario puede resetear password infinitas veces
→ Atacante prueba millones de tokens
- ✓ BIEN: Rate limiting + Expiración + Hash
 1. Máximo 3 intentos por hora (rate limiting)
 2. Token expira en 15 minutos
 3. Almacenar hash del token (no texto plano)

Mensajes de Error Genéricos:

- ✗ MAL: "User not found" vs "Wrong password"
→ Atacante sabe si email existe
 - ✓ BIEN: "Invalid credentials" (siempre el mismo)
→ Atacante NO sabe qué falló
-

A05: Configuración de Seguridad Incorrecta (Security Misconfiguration) (1/2)

El Problema:

Credenciales por defecto (admin/admin)
Stack traces en producción
Servicios innecesarios habilitados
Versiones desactualizadas
CORS mal configurado

A05: Configuración de Seguridad Incorrecta (Security Misconfiguration) (2/2)

✗ Vulnerable:

```
// CORS abierto a todos
app.use(
  cors({
    origin: '*', // Cualquiera puede acceder!
    credentials: true,
  })
)

// Errores detallados en producción
app.use((err, req, res, next) => {
  res.status(500).json({
    error: err.message,
    stack: err.stack, // Expone rutas internas!
  })
})

// Modo debug en producción
const app = express()
app.set('env', 'development') // Debug habilitado
```

A05: Defensa

4 Reglas de Configuración Segura:

1. CORS Restringido

- ✗ MAL: origin: '*' (cualquier sitio puede acceder)
- ✓ BIEN: origin: ['tudominio.com'] (solo tus dominios)

2. Errores Genéricos en Producción

- ✗ MAL: Mostrar stack trace al cliente
 - Expone rutas internas, versiones de librerías
- ✓ BIEN: "Internal server error" (genérico)
 - Loggear error completo solo en servidor

3. Security Headers (usar Helmet.js)

X-Frame-Options, CSP, HSTS, etc.

4. Actualizaciones Regulares

npm audit fix (mensual)

A07: Fallas de Autenticación (Authentication Failures) (1/2)

El Problema:

Passwords débiles permitidos
Sin MFA (autenticación multifactor)
Session fixation (atacante fuerza ID de sesión conocido)
Vulnerable a credential stuffing (reutilización de passwords robados)

A07: Fallas de Autenticación (Authentication Failures) (2/2)

✗ Vulnerable:

```
// Sin requisitos de password
app.post('/register', async (req, res) => {
  const user = await db.users.create({
    email: req.body.email,
    password: req.body.password, // "123" aceptado!
  })
})

// Sesiones permanentes
res.cookie('session', sessionId, {
  maxAge: 365 * 24 * 60 * 60 * 1000, // 1 año!
})

// Sin bloqueo de cuenta
// Usuario puede probar passwords ilimitados
```

A07: Defensa

3 Pilares de Autenticación Segura:

1. Passwords Fuertes

- ✗ MAL: Permitir "123456" (6 caracteres)
- ✓ BIEN: Validar con Zod
 - Mínimo 12 caracteres
 - Al menos 1 mayúscula
 - Al menos 1 número
 - Al menos 1 carácter especial

2. Sesiones de Corta Duración

- ✗ MAL: Cookie válida por 1 año
- ✓ BIEN: Expiración en 30 minutos
 - + httpOnly: true (protección XSS)
 - + sameSite: 'strict' (protección CSRF)

3. Bloqueo de Cuenta

Después de 5 intentos fallidos → bloquear 15 minutos
Previene brute force attacks

A09: Fallas en Logging y Monitoreo (Logging & Monitoring Failures) (1/2)

El Problema:

Sin logs de eventos críticos
Logs sin alertas
Logs exponen datos sensibles
Sin respuesta a incidentes (incident response)

A09: Fallas en Logging y Monitoreo (Logging & Monitoring Failures) (2/2)

✗ Logging Incorrecto:

```
// Sin logging
app.post('/login', async (req, res) => {
  const user = await authenticateUser(req.body)
  res.json({ token: createToken(user) })
  // No registro de quién hizo login!
})

// Logging de passwords
console.log('Login attempt:', {
  email: req.body.email,
  password: req.body.password, // NUNCA loguear passwords!
```

})

A09: Defensa (1/2)

3 Reglas de Logging Seguro:

1. Loguear Eventos de Seguridad



Loguear:

- Login exitoso: userId, email, IP, timestamp
- Login fallido: email, IP, reason (NO password!)
- Cambios de permisos
- Acceso a recursos sensibles



NUNCA loguear:

- Passwords
- Tokens completos
- Tarjetas de crédito
- SSN o datos médicos

2. Alertas Automáticas

Si >10 intentos fallidos → Sentry alert
"Possible brute force attack"

A09: Defensa (2/2)

3. Respuesta a Incidentes

Logs permiten investigar:

¿Quién? → userId, IP

¿Cuándo? → timestamp

¿Qué? → action (login, delete, etc.)

Checklist de Seguridad (cont.) (1/2)

Infraestructura:

- ✓ Security headers (Helmet)
- ✓ CORS restrictivo
- ✓ Protección CSRF
- ✓ Dependencias actualizadas (npm audit)

Monitoreo:

- ✓ Sentry para errores
- ✓ Intentos de login logueados
- ✓ Alertas de autenticación fallida
- ✓ Monitoreo de performance

Checklist de Seguridad (cont.) (2/2)

Mejores Prácticas:

- ✓ Principio de mínimo privilegio (least privilege)
- ✓ Defensa en profundidad (defense in depth)
- ✓ Fallar de forma segura (fail securely)
- ✓ Revisiones de seguridad regulares

Ejercicio 1: Prevenir SQL Injection

Prompt:

Actúa como security engineer mitigando vulnerabilidad A03 (Injection) de OW

CONTEXT0: SQL Injection = atacante inyecta código SQL en input de usuario q en database. String concatenation vulnerable: `SELECT * FROM products WHERE permite inyectar 'OR 1=1' → retorna TODAS las filas. Prepared statements (q parametrizadas): placeholders (?) que db driver escapa automáticamente (con en literal string, NO código ejecutable). ORMs (Prisma, TypeORM): generan p statements automáticamente. Tipos de injection: SQL, NoSQL, LDAP, OS **comman** NUNCA concatenar user input en queries, siempre usar parametrización. Segun validación de input con Zod (rechaza caracteres sospechosos). Principio de Privilegio: db user con permisos mínimos (solo SELECT, NO DROP TABLE).

TAREA: Reemplaza query vulnerable con prepared statement para prevenir SQL

UBICACIÓN:

- Archivo: src/api/mockServer.ts (o donde tengas queries)
- Endpoint: GET /products/:id (o similar endpoint que busca por ID)
- Query vulnerable: Template string con `${req.params.id}`

IMPLEMENTACIÓN SEGURA:

- Patrón: Prepared statement con placeholder ?
- Parámetros: Array [req.params.id] pasado separadamente
- Driver escapa: Automáticamente trata input como data, NO código

PATRÓN PREPARED STATEMENT:

```
// ❌ VULNERABLE
const query = `SELECT * FROM products WHERE id = ${req.params.id}`
```

```
// ✅ SEGURO
const query = 'SELECT * FROM products WHERE id = ?'
db.query(query, [req.params.id])
```

TESTING SQL INJECTION:

1. Request normal: GET /products/1
 - Debe retornar producto con id 1 ✅
2. Intento de injection: GET /products/1%20OR%201=1
 - Vulnerable: retorna TODOS los productos ❌
 - Seguro: retorna error "Product not found" ✅
3. Otro intento: GET /products/1';DROP TABLE products;--
 - Vulnerable: BORRA tabla products ❌❌❌
 - Seguro: busca literal string "1';DROP..." → not found ✅

VALIDACIÓN: Intentos de injection deben fallar sin afectar database

Aprende: Prepared statements tratan input de usuario como datos,

nunca como código ejecutable

Ejercicio 2: Rate Limiting para Prevenir Abuse

Prompt:

Actúa como security architect implementando rate limiting contra A04 (Diseño y A07 (Fallas de Autenticación)).

CONTEXT0: Rate limiting = límite de requests por tiempo por IP/usuario (pre Sin rate limiting: atacante hace 1000 intentos de login/segundo (brute force) express-rate-limit: middleware que trackea requests por IP + bloquea si excede windowMs: ventana de tiempo (60000 = 1 minuto). max: requests máximos en ventana requests/minuto). HTTP 429 Too Many Requests: código de estado cuando se excede Por qué importa: credential stuffing (atacante prueba millones de passwords) DDoS (atacante sobrecarga servidor), scraping (extracción masiva de datos). profundidad: rate limiting + CAPTCHA + bloqueo de cuenta (múltiples capas). endpoint: endpoints sensibles (login, reseteo de password) tienen límites M que endpoints públicos.

TAREA: Agrega rate limiting al endpoint /api/cart para prevenir abuse.

INSTALACIÓN:

- Comando: `pnpm add express-rate-limit`
- Librería: Middleware de limitación de requests

IMPLEMENTACIÓN:

- Archivo: `src/api/mockServer.ts`
- Ubicación: ANTES de route handlers (middleware debe ejecutar primero)
- Target: /api/cart endpoints (GET, POST, DELETE)

CONFIGURACIÓN RATE LIMIT:

```
import rateLimit from 'express-rate-limit'

const cartLimiter = rateLimit({
  windowMs: 60 * 1000, // 1 minuto
  max: 5, // 5 requests por minuto por IP
  message: 'Too many requests, please try again later'
})

app.use('/api/cart', cartLimiter)
```

TESTING RATE LIMIT:

1. Abrir DevTools Console
2. Ejecutar script rápido:

```
for (let i = 1; i <= 6; i++) {
  fetch('/api/cart')
    .then(r => console.log(`Request ${i}: ${r.status}`))
}
```

3. Verificar output:

- Request 1-5: 200 OK 
- Request 6: 429 Too Many Requests 
- Response body: "Too many requests, please try again later"

4. Esperar 1 minuto + reintentar → debe funcionar de nuevo (ventana resetea

VALIDACIÓN: Request #6 dentro de 1 minuto debe fallar con 429

Aprende: Rate limiting previene brute force,

credential stuffing y abuse de endpoints

Puntos Clave

1. **OWASP Top 10:** Estándar para vulnerabilidades web
2. **Control de Acceso:** Validar en backend siempre
3. **Criptografía:** bcrypt para passwords, HTTPS en todas partes
4. **Inyección:** Queries parametrizadas, validación de input
5. **Diseño Seguro:** Rate limiting, defensa en profundidad
6. **Configuración:** Helmet, CORS restrictivo, sin debug en producción
7. **Autenticación:** Passwords fuertes, sesiones cortas
8. **Monitoreo:** Loguear eventos de seguridad, alertar sobre actividad sospechosa
9. **Defensa en Profundidad:** Múltiples capas de seguridad
10. **Auditorías Regulares:** npm audit, revisiones de código, pruebas de penetración